Data Analysis & Visualisation

Midterm Comprehensive Notes

Covers: Google Sheets · Python · NumPy · Pandas

Analytics Lifecycle · Data Cleaning · Visualisation

Topics Covered

- Data Analysis & Visualisation
- Types of Analytics
- Analytics Lifecycle
- Google Sheets Basics
- Cell Referencing
- Data Types & Cleaning
- Sorting & Filtering
- Functions (all types)
- Lookup Functions
- Pivot Tables & Charts
- What-If Analysis
- Dashboard Creation
- NumPy Arrays & Ops
- Pandas Data Processing

March 11, 2026

0 1 Contents

1 Data Analysis and Visualisation — Overview **2**
1.1 Why Data Analysis Matters . . . 2

2 Types of Analytics **2**
2.1 Descriptive Analytics . . . 2
2.2 Diagnostic Analytics . . . 2
2.3 Predictive Analytics . . . 2
2.4 Prescriptive Analytics . . . 3

3 Analytics Lifecycle **3**

4 Google Sheets — Introduction & Interface **3**
4.1 Key Interface Components . . . 3
4.2 Cell Referencing . . . 4

5 Data Types in Google Sheets **4**

6 Sorting and Filtering **4**
6.1 Sorting . . . 4
6.2 Filtering . . . 4

7 Data Cleaning and Standardisation **5**
7.1 Handling Missing Values . . . 5
7.2 Find & Replace . . . 5
7.3 Text to Columns . . . 5
7.4 Text Standardisation Functions . . . 5

8 Functions **6**
8.1 Number Functions . . . 6
8.2 String (Text) Functions . . . 6
8.3 Date & Time Functions . . . 6
8.4 Logical Functions . . . 7
8.5 Aggregate Functions . . . 7
8.6 Conditional Aggregations . . . 7

9 Conditional Formatting **7**

10 Lookup Functions **8**
10.1 VLOOKUP . . . 8
10.2 HLOOKUP . . . 8
10.3 XLOOKUP . . . 9
10.4 INDEX-MATCH . . . 9

11 Error Understanding and Handling **9**

12 Pivot Tables and Charts **10**
12.1 Pivot Tables . . . 10
12.2 Charts . . . 10

13 Array Formulas **10**

14 What-If Analysis **11**
14.1 Goal Seek . . . 11
14.2 Scenario Manager . . . 11

15 Dashboards and Report Creation **11**
15.1 Dashboard Principles . . . 11
15.2 Slicers . . . 12

16 Introduction to Python Libraries **12**

17 NumPy — Comprehensive Guide **12**
17.1 Arrays . . . 12
17.2 Indexing & Slicing . . . 13
17.3 Boolean Masking . . . 13
17.4 Vector Operations . . . 13
17.5 Numerical Operations . . . 14
17.6 Matrix Operations . . . 14

18 Pandas — Comprehensive Guide **15**
18.1 Data Loading . . . 15
18.2 Data Analysing and Cleaning . . . 15
18.3 Data Filtering . . . 16
18.4 Data Grouping . . . 17
18.5 Data Merging . . . 17
18.6 Data Pivoting . . . 18

19 NumPy & Pandas Preprocessing Cheat Sheet **18**
19.1 Complete NumPy Command Reference . . . 18
19.2 Complete Pandas Command Reference . . . 20

20 Matplotlib & Seaborn — Quick Reference **22**

21 Quick Reference Summary Tables **23**
21.1 Google Sheets Function Summary . . . 23
21.2 NumPy vs Pandas Comparison . . . 24

1 1 Data Analysis and Visualisation — Overview

What is Data Analysis?

Data analysis is the process of inspecting, cleaning, transforming, and modelling data with the goal of discovering useful information, drawing conclusions, and supporting decision-making.

What is Data Visualisation?

Data visualisation is the graphical representation of data and information using charts, graphs, maps, and dashboards to make complex data easier to understand and interpret.

1.1 1Why Data Analysis Matters

- Converts raw data into actionable insights
- Identifies trends, patterns, and anomalies
- Reduces uncertainty in decision-making
- Enables performance measurement and forecasting
- Supports evidence-based strategy

2 1 Types of Analytics

There are four main types of analytics, forming a spectrum from describing the past to prescribing the future:

Type	Question Answered	Example	Complexity
Descriptive	What happened?	Monthly sales report	Low
Diagnostic	Why did it happen?	Root cause of sales drop	Medium
Predictive	What will happen?	Sales forecasting	High
Prescriptive	What should we do?	Optimal pricing strategy	Very High

2.1 1Descriptive Analytics

Summarises historical data using statistics. Tools: averages, counts, aggregations, dashboards, reports.

2.2 1Diagnostic Analytics

Drill-down analysis to identify root causes. Tools: filtering, segmentation, correlation analysis.

2.3 1Predictive Analytics

Uses statistical models and ML to forecast future outcomes. Tools: regression, time series, machine learning.

2.4 1Prescriptive Analytics

Recommends actions to achieve desired outcomes. Tools: optimisation algorithms, simulation, decision trees.

3 1 Analytics Lifecycle

The Analytics Lifecycle Stages

1. **Problem Definition** — Define the business question, objectives, and success criteria.
2. **Data Collection** — Gather data from internal/external sources (databases, APIs, files, surveys).
3. **Data Preprocessing / Cleaning** — Handle missing values, duplicates, inconsistencies, wrong data types.
4. **Exploratory Data Analysis (EDA)** — Understand distributions, correlations, outliers using statistics and charts.
5. **Data Modelling / Analysis** — Apply statistical methods, models, formulas to extract insights.
6. **Visualisation & Reporting** — Communicate findings via dashboards, charts, reports.
7. **Decision & Action** — Stakeholders act on insights; measure outcomes.
8. **Iteration** — Refine based on feedback and new data.

4 1 Google Sheets — Introduction & Interface

4.1 1Key Interface Components

- **Spreadsheet**: Grid of rows (numbered) and columns (lettered)
- **Cell**: Intersection of row and column, e.g., A1, B3
- **Name Box**: Shows current cell reference (top-left)
- **Formula Bar**: Displays/edits content of active cell
- **Sheets Tab**: Multiple sheets within one workbook
- **Toolbar**: Formatting, alignment, number format options

4.2 1Cell Referencing

Types of Cell References

Type	Example	Behaviour
Relative	`=A1`	Both row and column change when copied
Absolute (both)	`=$A$1`	Neither row nor column changes
Absolute column	`=$A1`	Column fixed, row changes
Absolute row	`=A$1`	Row fixed, column changes

Note

Use **F4** to toggle between reference types when editing a formula. Absolute references are essential when copying formulas that reference a fixed lookup table.

5 1 Data Types in Google Sheets

Data Type	Examples	Notes
Number	42, 3.14, -100	Right-aligned by default
Text/String	"Hello", "NYC"	Left-aligned by default
Date	01/01/2024	Stored as serial numbers
Time	10:30 AM	Fraction of a day
Boolean	TRUE, FALSE	Result of logical tests
Error	#N/A, #DIV/0!	Indicates formula problems

6 1 Sorting and Filtering

6.1 1Sorting

- **Single column sort**: Data → Sort range → choose column and order (A-Z or Z-A)
- **Multi-column sort**: Add multiple sort levels (Sort by A, then by B)
- **Custom sort**: Sort by custom order (e.g., Mon, Tue, Wed...)
- Always check "Data has header row" to protect column names

6.2 1Filtering

- **Basic Filter**: Data → Create a filter. Click dropdown arrows to filter by value.
- **Filter by condition**: Greater than, less than, text contains, is empty, etc.
- **Filter Views**: Save different filter configurations without changing the underlying data.
- **FILTER function**: `=FILTER(range, condition1, [condition2])`

Listing 1: FILTER function examples

```
=FILTER(A2:C100, B2:B100="Sales")          -- Filter where dept = Sales
```

```
=FILTER(A2:C100, C2:C100>50000)            -- Filter where salary > 50000
=FILTER(A2:C100, (B2:B100="Sales")*(C2:C100>50000))  -- AND condition
```

7 1 Data Cleaning and Standardisation

7.1 1Handling Missing Values

- **Identify**: Use `COUNTBLANK(range)` or filter by "is empty"
- **Fill with default**: Replace blanks with 0, "N/A", or mean value
- **IFERROR / IF(ISBLANK)**: Handle missing values in formulas
- **Delete rows**: When missing data makes the row unusable

Listing 2: Missing value handling

```
=IF(ISBLANK(A2), "Unknown", A2)            -- Replace blank with text
=IFERROR(A2/B2, 0)                         -- Replace error with 0
=COUNTBLANK(A2:A100)                       -- Count blank cells
```

7.2 1Find & Replace

- **Shortcut**: Ctrl+H (Windows) / Cmd+H (Mac)
- Replace misspellings, standardise codes, fix inconsistencies
- Options: Match case, Match entire cell contents, Search using regex

7.3 1Text to Columns

Split data in one cell into multiple columns:

- Not natively "Text to Columns" in Sheets, but use: Data → Split text to columns
- Splits by comma, space, semicolon, or custom separator
- Useful for: splitting full names, addresses, date+time combos

7.4 1Text Standardisation Functions

```
=TRIM(A2)             -- Remove extra spaces
=UPPER(A2)            -- Convert to UPPERCASE
=LOWER(A2)            -- Convert to lowercase
=PROPER(A2)           -- Title Case
=SUBSTITUTE(A2," ","-")  -- Replace spaces with hyphens
=CLEAN(A2)            -- Remove non-printable characters
```

8 1 Functions

8.1 1Number Functions

Listing 3: Number functions

```
=ROUND(3.14159, 2)          -- 3.14
=ROUNDUP(3.1, 0)            -- 4
=ROUNDDOWN(3.9, 0)          -- 3
=INT(4.9)                   -- 4 (floor)
=ABS(-5)                    -- 5
=MOD(10, 3)                 -- 1 (remainder)
=POWER(2, 10)               -- 1024
=SQRT(144)                  -- 12
=CEILING(4.2, 1)            -- 5 (round up to nearest 1)
=FLOOR(4.8, 1)              -- 4 (round down to nearest 1)
=RAND()                     -- Random number 0 to 1
=RANDBETWEEN(1, 100)        -- Random integer between 1 and 100
```

8.2 1String (Text) Functions

Listing 4: String functions

```
=LEN("Hello")               -- 5 (length)
=LEFT("Hello", 3)           -- "Hel"
=RIGHT("Hello", 2)          -- "lo"
=MID("Hello", 2, 3)         -- "ell" (start, length)
=FIND("l", "Hello")         -- 3 (case-sensitive position)
=SEARCH("l", "Hello")       -- 3 (case-insensitive position)
=CONCATENATE(A1, " ", B1)   -- Join strings (or use &)
=A1&" "&B1                  -- Concatenation with &
=SPLIT("a,b,c", ",")        -- Split into multiple cells
=TEXTJOIN(", ", TRUE, A1:A5)-- Join range with delimiter
=REPT("*", 5)               -- "*****"
=VALUE("123")               -- Convert text "123" to number 123
=TEXT(1234.5, "#,##0.00")   -- "1,234.50"
```

8.3 1Date & Time Functions

Listing 5: Date and time functions

```
=TODAY()                    -- Current date
=NOW()                      -- Current date and time
=DATE(2024, 12, 25)         -- Create a date
=DAY(A1)                    -- Extract day number
=MONTH(A1)                  -- Extract month number
=YEAR(A1)                   -- Extract year
=WEEKDAY(A1, 2)             -- Day of week (1=Mon with mode 2)
=DATEDIF(A1, B1, "D")       -- Days between two dates
=DATEDIF(A1, B1, "M")       -- Months between dates
=DATEDIF(A1, B1, "Y")       -- Years between dates
=EDATE(A1, 3)               -- Date 3 months from A1
=EOMONTH(A1, 0)             -- Last day of month
=NETWORKDAYS(A1, B1)        -- Working days between dates
=TEXT(TODAY(), "dddd")      -- Day name (e.g., "Monday")
```

8.4 1Logical Functions

Listing 6: Logical functions

```
=IF(A1>100, "High", "Low")
=IF(A1>100, "High", IF(A1>50, "Medium", "Low"))  -- Nested IF
=AND(A1>0, A1<100)           -- TRUE if both conditions true
=OR(A1="Yes", A1="Y")        -- TRUE if any condition true
=NOT(A1="No")                -- Reverses TRUE/FALSE
=IFS(A1>90,"A", A1>80,"B", A1>70,"C", TRUE,"F")  -- Multiple conditions
=SWITCH(A1, 1,"Jan", 2,"Feb", 3,"Mar", "Other")  -- Like CASE statement
=IFERROR(formula, "Error")  -- Return value if formula errors
=IFNA(formula, "Not Found") -- Return value if #N/A error
```

8.5 1Aggregate Functions

Listing 7: Aggregate functions

```
=SUM(A1:A100)
=AVERAGE(A1:A100)
=MIN(A1:A100)
=MAX(A1:A100)
=COUNT(A1:A100)              -- Count numbers only
=COUNTA(A1:A100)             -- Count non-empty cells
=COUNTBLANK(A1:A100)         -- Count empty cells
=MEDIAN(A1:A100)
=MODE(A1:A100)
=STDEV(A1:A100)              -- Sample standard deviation
=STDEVP(A1:A100)             -- Population standard deviation
=VAR(A1:A100)                -- Variance
=LARGE(A1:A100, 3)           -- 3rd largest value
=SMALL(A1:A100, 2)           -- 2nd smallest value
=PERCENTILE(A1:A100, 0.75)   -- 75th percentile
```

8.6 1Conditional Aggregations

Listing 8: Conditional aggregation functions

```
-- COUNTIF / COUNTIFS
=COUNTIF(B2:B100, "Sales")
=COUNTIFS(B2:B100,"Sales", C2:C100,">50000")

-- SUMIF / SUMIFS
=SUMIF(B2:B100, "Sales", C2:C100)
=SUMIFS(C2:C100, B2:B100,"Sales", D2:D100,"2024")

-- AVERAGEIF / AVERAGEIFS
=AVERAGEIF(B2:B100, "Sales", C2:C100)
=AVERAGEIFS(C2:C100, B2:B100,"Sales", D2:D100,">2020")

-- Syntax: =SUMIFS(sum_range, criteria_range1, criteria1, ...)
```

9 1 Conditional Formatting

Conditional formatting automatically applies cell styles (color, bold, etc.) based on values or conditions.

- **How to apply**: Format → Conditional formatting → Set rules
- **Single color**: Apply one color if condition is met
- **Color scale**: Gradient scale (e.g., red for low, green for high)
- **Custom formula**: Use a formula for complex rules

Listing 9: Custom formula conditional formatting examples

```
=$C2>100                        -- Highlight row if C > 100
=$B2="Overdue"                  -- Highlight if text matches
=AND($C2>50, $D2="Active")      -- Highlight with multiple conditions
=$C2<AVERAGE($C$2:$C$100)       -- Below average values
```

Note

In custom formulas, use $ to lock the column but leave the row flexible so the rule applies to each row independently.

10 1 Lookup Functions

10.1 1VLOOKUP

VLOOKUP Syntax

```
=VLOOKUP(search_key, range, index, [is_sorted])
```

- **search_key**: Value to look for
- **range**: Table where to search (first column = lookup column)
- **index**: Column number to return (1 = first column of range)
- **is_sorted**: FALSE for exact match, TRUE for approximate

Listing 10: VLOOKUP examples

```
=VLOOKUP("E001", A2:D100, 3, FALSE)     -- Exact match, return col 3
=VLOOKUP(A2, $F$2:$H$50, 2, FALSE)     -- With absolute reference
```

Limitations: Only looks to the right; lookup column must be the first column.

10.2 1HLOOKUP

HLOOKUP Syntax

```
=HLOOKUP(search_key, range, index, [is_sorted])
```

Same as VLOOKUP but searches *horizontally* (across rows). Used when data is arranged in rows.

10.3 1XLOOKUP

XLOOKUP Syntax

```
=XLOOKUP(search_key, lookup_array, return_array, [if_not_found], [match_mode],
[search_mode])
```

Listing 11: XLOOKUP examples

```
=XLOOKUP("E001", A2:A100, C2:C100)          -- Basic lookup
=XLOOKUP("E001", A2:A100, C2:C100, "N/A")   -- With fallback
=XLOOKUP("E001", A2:A100, B2:D100)          -- Return multiple columns
=XLOOKUP(A2, A2:A100, B2:B100,,-1)          -- -1 = exact or next smaller
```

Advantages over VLOOKUP: Can look left, returns arrays, has built-in error handling, no column index needed.

10.4 1INDEX-MATCH

INDEX-MATCH Syntax

```
=INDEX(return_range, MATCH(search_key, lookup_range, 0))
```

Listing 12: INDEX-MATCH examples

```
=INDEX(C2:C100, MATCH("E001", A2:A100, 0))        -- Basic
=INDEX(A2:D100, MATCH("E001",A2:A100,0), 3)       -- Return from specific
   column
-- Two-way lookup:
=INDEX(B2:E10, MATCH("Row_val",A2:A10,0), MATCH("Col_val",B1:E1,0))
```

Why use INDEX-MATCH? More flexible than VLOOKUP: can look in any direction, not limited by column position, generally faster on large datasets.

11 1 Error Understanding and Handling

Common Spreadsheet Errors

Error	Meaning	Common Cause & Fix
`#DIV/0!`	Division by zero	Denominator is 0 or blank. Use `IFERROR` or `IF(B1=0,0,A1/B1)`
`#VALUE!`	Wrong data type	Math on text. Check data types, use `VALUE()`
`#REF!`	Invalid reference	Deleted referenced cells. Check formula references
`#N/A`	Value not found	VLOOKUP failed. Use `IFNA` or `IFERROR`
`#NAME?`	Unknown function	Typo in function name. Check spelling
`#NUM!`	Invalid number	e.g., SQRT of negative. Check input data
`#NULL!`	Invalid range	Missing colon in range. Fix syntax

Listing 13: Error handling

```
=IFERROR(formula, "Error Occurred")      -- Catch any error
```

```
=IFNA(VLOOKUP(A1, B:C, 2, 0), "N/A")    -- Catch only #N/A
=ISERROR(formula)                        -- Returns TRUE if error
=ISERR(formula)                          -- Returns TRUE if error (not #N/A)
=ISNA(formula)                           -- Returns TRUE if #N/A
=ISNUMBER(A1)                            -- Check if value is a number
```

12 1 Pivot Tables and Charts

12.1 1Pivot Tables

Pivot tables summarise large datasets by grouping, aggregating, and restructuring data.

Creating a Pivot Table in Google Sheets:

1. Select data range → Insert → Pivot table
2. Choose new sheet or existing sheet
3. In the Pivot table editor: Add Rows, Columns, Values, Filters

Key settings:

- **Rows**: Categories to group data by (e.g., Region, Product)
- **Columns**: Cross-tabulate by a second dimension (e.g., Year)
- **Values**: Numeric field to aggregate (SUM, COUNT, AVERAGE, MAX, etc.)
- **Filters**: Limit the data included in the pivot

12.2 1Charts

Chart Type	Best Used For
Bar / Column	Comparing categories
Line	Trends over time
Pie / Doughnut	Part-to-whole relationships (few categories)
Scatter	Correlation between two variables
Area	Cumulative trends over time
Histogram	Distribution of a single variable
Combo	Comparing different scales on same chart

Insert Chart: Select data → Insert → Chart → Customise in Chart editor

13 1 Array Formulas

Array Formula

An array formula performs operations on multiple values simultaneously and can return a single result or multiple results. In Google Sheets, press **Ctrl+Shift+Enter** or use the `ARRAYFORMULA()` function.

Listing 14: Array formula examples

```
=ARRAYFORMULA(A2:A100 * B2:B100)        -- Multiply two columns
=ARRAYFORMULA(IF(A2:A100>50,"High","Low")) -- Apply IF to range
=ARRAYFORMULA(VLOOKUP(A2:A100, D:E, 2, 0)) -- VLOOKUP for whole column
=SUM(ARRAYFORMULA(A2:A10 * B2:B10))     -- Sum of products
-- Spill arrays (Google Sheets auto-fills adjacent cells)
```

14 1 What-If Analysis

14.1 1Goal Seek

Goal Seek finds the input value needed to achieve a desired output.

- **Path**: Extensions → Goal Seek (or use add-on)
- **Example**: What sales volume is needed to achieve $1M profit?
- Set cell: The result cell (e.g., profit formula)
- To value: Desired result (e.g., 1000000)
- By changing cell: The input variable (e.g., units sold)

14.2 1Scenario Manager

Scenario Manager allows you to save and compare multiple sets of input values.

- Create named scenarios (Best Case, Worst Case, Base Case)
- Each scenario stores different values for changing cells
- Generate a summary report comparing scenarios
- Useful for: budget planning, business modelling, sensitivity analysis

Note

In Google Sheets, advanced What-If analysis (like Excel's Scenario Manager) may require the **Solver** add-on or manual data tables. Excel has more native support for these features.

15 1 Dashboard and Report Creation

15.1 1Dashboard Principles

- **Single view**: All key metrics visible without scrolling
- **Hierarchy**: Most important KPIs prominent (top-left)
- **Consistency**: Uniform colors, fonts, and chart styles
- **Interactivity**: Filters and slicers for user exploration
- **Context**: Include targets, benchmarks, and period comparisons

15.2 1Slicers

Slicers are visual filter buttons connected to pivot tables and charts.

- **Add slicer**: Click pivot table → Data → Add a slicer
- Select the column/field to filter by
- Multiple slicers can filter the same pivot table
- Clicking slicer buttons instantly filters all connected charts/pivots

16 1 Introduct to Python Libraries

Key Python Data Science Libraries

Library	Primary Use	Key Features
NumPy	Numerical computing	N-dimensional arrays, math ops, linear algebra
Pandas	Data manipulation	DataFrames, data cleaning, groupby, merge
Matplotlib	Plotting	Line, bar, scatter, histogram, subplots
Seaborn	Statistical viz	Beautiful stats charts, built on Matplotlib

Listing 15: Standard imports

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

17 1 NumPy — Comprehensive Guide

17.1 1Arrays

Listing 16: Creating NumPy arrays

```python
import numpy as np

# From lists
arr1d = np.array([1, 2, 3, 4, 5])
arr2d = np.array([[1, 2, 3], [4, 5, 6]])

# Special arrays
np.zeros((3, 4))           # 3x4 array of zeros
np.ones((2, 3))            # 2x3 array of ones
np.full((3, 3), 7)         # 3x3 array filled with 7
np.eye(4)                  # 4x4 identity matrix
np.arange(0, 10, 2)        # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)       # [0, 0.25, 0.5, 0.75, 1.0]
np.random.rand(3, 3)       # 3x3 random floats [0,1)
np.random.randint(0,100,(3,3)) # Random integers

```

```
# Array properties
arr2d.shape       # (2, 3)
arr2d.ndim        # 2
arr2d.size        # 6
arr2d.dtype       # dtype('int64')

# Reshaping
arr1d.reshape(5, 1)        # Column vector
arr2d.flatten()            # 1D array
arr2d.T                    # Transpose
```

17.2 1Indexing & Slicing

Listing 17: NumPy indexing and slicing

```
arr = np.array([10, 20, 30, 40, 50])
mat = np.array([[1,2,3],[4,5,6],[7,8,9]])

# 1D indexing
arr[0]            # 10 (first element)
arr[-1]           # 50 (last element)
arr[1:4]          # [20, 30, 40] (slice)
arr[::2]          # [10, 30, 50] (every 2nd)

# 2D indexing
mat[0, 1]         # 2 (row 0, col 1)
mat[1, :]         # [4, 5, 6] (entire row 1)
mat[:, 2]         # [3, 6, 9] (entire col 2)
mat[0:2, 1:3]     # Sub-matrix rows 0-1, cols 1-2

# Fancy indexing
arr[[0, 2, 4]]    # [10, 30, 50] (select by indices)
mat[[0,2], :]     # Rows 0 and 2
```

17.3 1Boolean Masking

Listing 18: Boolean masking

```
arr = np.array([10, 25, 3, 47, 8, 30])

# Create boolean mask
mask = arr > 20           # [F, T, F, T, F, T]

# Apply mask (filter)
arr[mask]                 # [25, 47, 30]
arr[arr > 20]             # Same, inline
arr[(arr > 10) & (arr < 40)]  # AND condition -> [25, 30]
arr[(arr < 5) | (arr > 40)]   # OR condition -> [3, 47]
arr[~(arr > 20)]          # NOT: values <= 20

# Replace values using mask
arr[arr < 10] = 0         # Set values < 10 to 0
np.where(arr > 20, arr, 0)     # Conditional replacement
```

17.4 1Vector Operations

Listing 19: Vector and element-wise operations

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

# Element-wise (vectorized operations)
a + b              # [11, 22, 33, 44]
a * b              # [10, 40, 90, 160]
a / b              # [0.1, 0.1, 0.1, 0.1]
a ** 2             # [1, 4, 9, 16]

# Broadcasting (scalar operations)
a + 100            # [101, 102, 103, 104]
a * 5              # [5, 10, 15, 20]

# Broadcasting between different shapes
col = np.array([[1],[2],[3]])  # (3,1)
row = np.array([10, 20, 30])   # (3,)
col + row          # (3,3) matrix (broadcasted)
```

17.5 1Numerical Operations

Listing 20: Statistical and mathematical operations

```
arr = np.array([3, 1, 4, 1, 5, 9, 2, 6])

# Statistics
np.sum(arr)                # 31
np.mean(arr)               # 3.875
np.median(arr)             # 3.5
np.std(arr)                # Standard deviation
np.var(arr)                # Variance
np.min(arr)                # 1
np.max(arr)                # 9
np.argmin(arr)             # Index of minimum
np.argmax(arr)             # Index of maximum
np.percentile(arr, 75)     # 75th percentile

# Sorting
np.sort(arr)               # Sorted copy
np.argsort(arr)            # Indices that sort the array
np.unique(arr)             # Unique values

# Cumulative operations
np.cumsum(arr)             # Cumulative sum
np.cumprod(arr)            # Cumulative product

# Math functions
np.sqrt(arr)
np.log(arr)                # Natural log
np.log10(arr)              # Log base 10
np.exp(arr)                # e^x
np.abs(arr)                # Absolute value
np.round(arr, 2)           # Round to 2 decimals
```

17.6 1Matrix Operations

Listing 21: Matrix operations

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

# Matrix multiplication
A @ B                       # Preferred syntax
np.dot(A, B)                # Same result
np.matmul(A, B)             # Same result

# Element-wise vs matrix multiply
A * B                       # Element-wise (NOT matrix mult)

# Linear algebra
np.linalg.det(A)            # Determinant
np.linalg.inv(A)            # Inverse matrix
np.linalg.eig(A)            # Eigenvalues and eigenvectors
np.linalg.norm(A)           # Matrix norm

A.T                         # Transpose
np.trace(A)                 # Sum of diagonal
np.diag(A)                  # Extract diagonal elements
```

18 1 Pandas — Comprehensive Guide

18.1 1Data Loading

Listing 22: Loading data with Pandas

```
import pandas as pd

# Read files
df = pd.read_csv("data.csv")
df = pd.read_csv("data.csv", sep=";")              # Semicolon delimiter
df = pd.read_csv("data.csv", encoding="utf-8")
df = pd.read_csv("data.csv", skiprows=2)           # Skip first 2 rows
df = pd.read_csv("data.csv", nrows=1000)           # Read only 1000 rows
df = pd.read_excel("data.xlsx", sheet_name="Sheet1")
df = pd.read_json("data.json")
df = pd.read_sql("SELECT * FROM table", conn)      # From SQL

# Quick inspection
df.head(5)                  # First 5 rows
df.tail(5)                  # Last 5 rows
df.shape                    # (rows, columns)
df.info()                   # Column names, dtypes, non-null counts
df.describe()               # Summary statistics
df.columns                  # Column names
df.dtypes                   # Data types per column
df.index                    # Row index
```

18.2 1Data Analysing and Cleaning

Listing 23: Data analysis and cleaning

```
# --- Missing Values ---
df.isnull().sum()               # Count nulls per column
df.isnull().sum() / len(df)     # Null percentage
```

```python

# Drop missing
df.dropna()                      # Drop rows with any null
df.dropna(subset=["salary"])     # Drop rows where salary is null
df.dropna(thresh=3)              # Drop rows with < 3 non-nulls

# Fill missing
df.fillna(0)                                # Fill all nulls with 0
df["age"].fillna(df["age"].mean())          # Fill with mean
df["city"].fillna("Unknown")               # Fill text with default
df.fillna(method="ffill")                  # Forward fill
df.fillna(method="bfill")                  # Backward fill

# --- Duplicates ---
df.duplicated().sum()            # Count duplicate rows
df[df.duplicated()]              # Show duplicate rows
df.drop_duplicates()             # Remove all duplicates
df.drop_duplicates(subset=["id"])   # Based on specific column
df.drop_duplicates(keep="last")     # Keep last occurrence

# --- Data Types ---
df.dtypes
df["age"] = df["age"].astype(int)
df["date"] = pd.to_datetime(df["date"])
df["salary"] = pd.to_numeric(df["salary"], errors="coerce")

# --- Inconsistent Data ---
df["gender"].str.lower()                # Standardise case
df["name"].str.strip()                  # Remove whitespace
df["city"].str.replace(" ", "_")       # Replace spaces
df["status"].map({"yes":1, "no":0})    # Map values
```

18.3 1Data Filtering

Listing 24: Filtering in Pandas

```python
# Boolean filtering
df[df["salary"] > 50000]
df[df["dept"] == "Sales"]
df[(df["salary"] > 50000) & (df["dept"] == "Sales")]
df[(df["age"] < 25) | (df["age"] > 60)]
df[~(df["dept"] == "HR")]               # NOT HR

# .loc and .iloc
df.loc[5]                               # Row by label/index
df.loc[0:5, "name":"salary"]            # Rows 0-5, cols name to salary
df.iloc[0:5, 0:3]                       # By position

# query() method
df.query("salary > 50000 and dept == 'Sales'")

# isin / ~isin
df[df["dept"].isin(["Sales","IT"])]
df[~df["dept"].isin(["HR"])]

# String conditions
df[df["name"].str.contains("John")]
df[df["city"].str.startswith("New")]
df[df["email"].str.endswith(".com")]

```

```python
# Between
df[df["age"].between(25, 40)]

# Filter by date
df[df["date"] > "2023-01-01"]
df[(df["date"] >= "2023-01-01") & (df["date"] <= "2023-12-31")]
```

18.4 1Data Grouping

Listing 25: GroupBy operations in Pandas

```python
# Basic groupby
df.groupby("dept")["salary"].mean()
df.groupby("dept")["salary"].agg(["mean","sum","count","max","min"])

# Multiple groupby keys
df.groupby(["dept","year"])["salary"].sum()

# Custom aggregations
df.groupby("dept").agg(
    avg_salary=("salary", "mean"),
    total_salary=("salary", "sum"),
    headcount=("id", "count"),
    max_salary=("salary", "max")
)

# Named aggregation with reset_index
result = df.groupby("dept")["salary"].mean().reset_index()

# Transform (keep original shape)
df["dept_avg"] = df.groupby("dept")["salary"].transform("mean")

# Filter groups
df.groupby("dept").filter(lambda x: x["salary"].mean() > 60000)

# Apply custom function
df.groupby("dept").apply(lambda g: g.nlargest(3, "salary"))
```

18.5 1Data Merging

Listing 26: Merging DataFrames in Pandas

```python
# merge() - like SQL JOIN
merged = pd.merge(df1, df2, on="id")                    # Inner join
merged = pd.merge(df1, df2, on="id", how="left")        # Left join
merged = pd.merge(df1, df2, on="id", how="right")       # Right join
merged = pd.merge(df1, df2, on="id", how="outer")       # Full outer join

# Merge on different column names
pd.merge(df1, df2, left_on="emp_id", right_on="id")

# Merge on multiple keys
pd.merge(df1, df2, on=["year", "dept"])

# concat() - stack DataFrames
pd.concat([df1, df2])                  # Stack vertically (rows)
pd.concat([df1, df2], axis=1)          # Stack horizontally (cols)
pd.concat([df1, df2], ignore_index=True)    # Reset index
```

```python

# join() - join on index
df1.join(df2, on="key", how="left")
```

SQL vs Pandas JOIN Equivalents

SQL	Pandas
INNER JOIN	`pd.merge(df1, df2, how="inner")`
LEFT JOIN	`pd.merge(df1, df2, how="left")`
RIGHT JOIN	`pd.merge(df1, df2, how="right")`
FULL OUTER JOIN	`pd.merge(df1, df2, how="outer")`
UNION ALL	`pd.concat([df1, df2])`
UNION DISTINCT	`pd.concat([df1,df2]).drop_duplicates()`

18.6 1Data Pivoting

Listing 27: Pivot and reshape in Pandas

```python
# pivot_table (like Google Sheets Pivot)
pt = df.pivot_table(
    values="sales",
    index="region",
    columns="year",
    aggfunc="sum",
    fill_value=0
)

# pivot (no aggregation, must have unique index-column pairs)
df.pivot(index="date", columns="product", values="price")

# melt (wide to long - unpivot)
pd.melt(df, id_vars=["region"], value_vars=["Q1","Q2","Q3","Q4"],
        var_name="Quarter", value_name="Sales")

# stack / unstack
df.stack()                # Columns to row index (long format)
df.unstack()              # Row index to columns (wide format)

# crosstab (frequency table)
pd.crosstab(df["gender"], df["dept"])
pd.crosstab(df["gender"], df["dept"], values=df["salary"], aggfunc="mean")
```

19 1 NumPy & Pandas Preprocessing Cheat Sheet

19.1 1Complete NumPy Command Reference

Listing 28: NumPy comprehensive reference

```python
import numpy as np

# ========== ARRAY CREATION ==========
np.array([1,2,3])                    # From list
np.zeros((m, n))                     # Zero matrix
np.ones((m, n))                      # Ones matrix
```

```python
np.eye(n)                              # Identity matrix
np.arange(start, stop, step)           # Range array
np.linspace(start, stop, num)          # Evenly spaced
np.random.seed(42)                     # Set seed for reproducibility
np.random.rand(m, n)                   # Uniform [0,1)
np.random.randn(m, n)                  # Standard normal
np.random.randint(low, high, size)     # Random integers
np.random.choice(arr, size, replace=False) # Random sample

# ========== ARRAY MANIPULATION ==========
arr.reshape(m, n)                      # Reshape
arr.flatten()                          # To 1D (copy)
arr.ravel()                            # To 1D (view if possible)
arr.T                                  # Transpose
np.concatenate([a, b], axis=0)         # Join arrays
np.vstack([a, b])                      # Vertical stack
np.hstack([a, b])                      # Horizontal stack
np.split(arr, 3)                       # Split into 3 equal parts
np.tile(arr, 3)                        # Repeat array 3 times
np.repeat(arr, 3)                      # Repeat each element 3 times
arr.squeeze()                          # Remove dimensions of size 1
np.expand_dims(arr, axis=0)            # Add new axis

# ========== INDEXING ==========
arr[2]                                 # Single element
arr[1:5]                               # Slice
arr[::2]                               # Every 2nd element
arr[-1]                                # Last element
arr2d[r, c]                            # Row r, column c
arr2d[r1:r2, c1:c2]                   # Submatrix
arr[arr > 0]                           # Boolean filter
arr[[0,2,4]]                           # Fancy indexing

# ========== STATISTICS ==========
np.sum(arr, axis=0)                   # Sum by column
np.sum(arr, axis=1)                   # Sum by row
np.mean(arr)
np.std(arr, ddof=1)                   # Sample std
np.var(arr, ddof=1)                   # Sample variance
np.min(arr); np.max(arr)
np.argmin(arr); np.argmax(arr)
np.median(arr)
np.percentile(arr, [25,50,75])
np.corrcoef(x, y)                      # Correlation matrix
np.cov(x, y)                           # Covariance matrix

# ========== MATH ==========
np.sqrt(arr)
np.log(arr); np.log2(arr); np.log10(arr)
np.exp(arr)
np.sin(arr); np.cos(arr); np.tan(arr)
np.ceil(arr); np.floor(arr)
np.abs(arr)
np.clip(arr, min_val, max_val)         # Clip to range
np.where(condition, x, y)              # Conditional replacement

# ========== LINEAR ALGEBRA ==========
A @ B                                  # Matrix multiply
np.linalg.inv(A)                       # Inverse
np.linalg.det(A)                       # Determinant
np.linalg.eig(A)                       # Eigenvalues/vectors
```

```
np.linalg.solve(A, b)                # Solve Ax = b
np.linalg.norm(v)                    # Vector norm

# ========== SORTING ==========
np.sort(arr)                         # Sorted copy (ascending)
np.sort(arr)[::-1]                   # Descending
np.argsort(arr)                      # Sort indices
np.unique(arr, return_counts=True)   # Unique + counts
```

19.2 1Complete Pandas Command Reference

Listing 29: Pandas comprehensive reference - Part 1: Basics and I/O

```
import pandas as pd

# ========== CREATE DATAFRAME ==========
df = pd.DataFrame({"A":[1,2,3], "B":[4,5,6]})
df = pd.DataFrame(data, columns=["col1","col2"])
s = pd.Series([1,2,3,4], name="MySeries")

# ========== I/O ==========
pd.read_csv("f.csv", sep=",", encoding="utf-8",
            parse_dates=["date"], index_col="id",
            usecols=["col1","col2"], nrows=1000)
df.to_csv("out.csv", index=False)
pd.read_excel("f.xlsx", sheet_name=0)
df.to_excel("out.xlsx", index=False)

# ========== INSPECTION ==========
df.shape; df.ndim; df.size
df.dtypes; df.info(); df.describe()
df.head(n); df.tail(n)
df.sample(n)                         # Random n rows
df.columns.tolist()
df.index.tolist()
df.memory_usage()
```

Listing 30: Pandas comprehensive reference - Part 2: Selection and Filtering

```
# ========== SELECTION ==========
df["col"]                            # Single column (Series)
df[["col1","col2"]]                  # Multiple columns (DataFrame)
df.loc[idx]                          # By label
df.loc[idx, "col"]                   # Label + column
df.loc[0:5, "A":"C"]                # Range by label
df.iloc[0]                           # By position
df.iloc[0:5, 0:3]                   # Position range
df.at[idx, "col"]                    # Fast single value
df.iat[0, 0]                         # Fast positional

# ========== FILTERING ==========
df[df["A"] > 5]
df.query("A > 5 and B < 10")
df[df["city"].isin(["NYC","LA"])]
df[df["name"].str.contains("Smith")]
df[df["date"].between("2023-01-01","2023-12-31")]
df[df["val"].notnull()]
df[df["val"].isnull()]
```

Listing 31: Pandas comprehensive reference - Part 3: Cleaning

```
# ========== MISSING VALUES ==========
df.isnull().sum()
df.notnull().sum()
df.dropna()
df.dropna(subset=["col"])
df.dropna(thresh=n)                 # Keep rows with >= n non-NaN
df.fillna(val)
df.fillna({"A":0, "B":"Unknown"}) # Different fills per column
df["A"].fillna(df["A"].median())
df.interpolate()                    # Interpolate numeric

# ========== DUPLICATES ==========
df.duplicated().sum()
df.drop_duplicates()
df.drop_duplicates(subset=["id"], keep="first")

# ========== TYPE CONVERSION ==========
df["col"].astype("int")
df["col"].astype("float")
df["col"].astype("str")
pd.to_numeric(df["col"], errors="coerce")
pd.to_datetime(df["col"], format="%Y-%m-%d")

# ========== STRING OPERATIONS ==========
df["col"].str.lower()
df["col"].str.upper()
df["col"].str.strip()
df["col"].str.replace("old","new")
df["col"].str.split(",")            # Split to list
df["col"].str.split(",", expand=True)  # Split to columns
df["col"].str.len()
df["col"].str.contains("pattern")
df["col"].str.extract(r"(\d+)")     # Regex extract

# ========== DATE OPERATIONS ==========
df["date"].dt.year
df["date"].dt.month
df["date"].dt.day
df["date"].dt.weekday               # 0=Monday
df["date"].dt.day_name()
df["date"].dt.days_in_month
```

Listing 32: Pandas comprehensive reference - Part 4: Transform and Aggregate

```
# ========== ADD/MODIFY COLUMNS ==========
df["new"] = df["A"] + df["B"]
df["label"] = df["score"].apply(lambda x: "Pass" if x>=50 else "Fail")
df["grade"] = np.where(df["score"]>=90,"A",
              np.where(df["score"]>=80,"B","C"))
df.assign(ratio=lambda x: x["A"]/x["B"])
df.rename(columns={"old":"new"})
df.drop(columns=["col1","col2"])
df.drop(index=[0,1,2])

# ========== GROUPBY ==========
g = df.groupby("dept")
g["salary"].mean()
g["salary"].agg(["mean","sum","count"])
g.agg({"salary":"sum", "age":"mean"})
g.apply(lambda x: x.nlargest(3,"salary"))
```

```python
g.transform("mean")                # Keep original size
g.filter(lambda x: len(x) > 5)    # Filter groups

# ========== PIVOT TABLES ==========
df.pivot_table(values="sales", index="region",
               columns="year", aggfunc="sum", fill_value=0)
pd.melt(df, id_vars=["id"],
        value_vars=["Q1","Q2","Q3"], var_name="Q", value_name="val")

# ========== MERGE / JOIN ==========
pd.merge(df1, df2, on="key", how="inner")
pd.merge(df1, df2, left_on="a", right_on="b")
pd.concat([df1, df2], axis=0, ignore_index=True)
df1.join(df2, on="key")

# ========== SORTING ==========
df.sort_values("salary", ascending=False)
df.sort_values(["dept","salary"], ascending=[True,False])
df.sort_index()
df.nlargest(5, "salary")
df.nsmallest(5, "age")

# ========== APPLY / MAP ==========
df["col"].apply(func)              # Apply function element-wise
df.apply(func, axis=1)            # Apply to each row
df.applymap(func)                  # Apply to every cell
df["col"].map({"a":1, "b":2})     # Map values
```

Listing 33: Pandas comprehensive reference - Part 5: Useful utility operations

```python
# ========== USEFUL UTILITIES ==========
df.value_counts()                  # Count unique values (Series)
df["col"].value_counts()
df["col"].unique()
df["col"].nunique()
df["col"].mode()
df["col"].corr(df["col2"])         # Correlation

# Window functions
df["rolling_avg"] = df["sales"].rolling(window=7).mean()
df["cumsum"] = df["sales"].cumsum()
df["pct_change"] = df["sales"].pct_change()
df["rank"] = df["salary"].rank(ascending=False)

# Reshaping
df.set_index("id")                 # Set column as index
df.reset_index()                   # Move index to column
df.stack(); df.unstack()
df.T                               # Transpose

# Output / export
df.to_csv("output.csv", index=False)
df.to_excel("output.xlsx", index=False)
df.to_dict()
df.to_numpy()                      # Convert to NumPy array
```

20 1 Matplotlib & Seaborn — Quick Reference

Listing 34: Matplotlib and Seaborn basics

```python
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Matplotlib ----
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(x, y, color="blue", label="Sales")        # Line chart
ax.bar(x, y, color="orange")                       # Bar chart
ax.scatter(x, y, color="green", alpha=0.6)         # Scatter plot
ax.hist(data, bins=20, color="purple")             # Histogram
ax.boxplot(data)                                   # Box plot
ax.pie(sizes, labels=labels, autopct="%1.1f%%")   # Pie chart

ax.set_title("Title"); ax.set_xlabel("X"); ax.set_ylabel("Y")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("chart.png", dpi=150)
plt.show()

# ---- Seaborn ----
sns.set_theme(style="whitegrid")

sns.lineplot(data=df, x="date", y="sales", hue="region")
sns.barplot(data=df, x="dept", y="salary", palette="Set2")
sns.scatterplot(data=df, x="age", y="salary", hue="gender")
sns.histplot(df["salary"], bins=30, kde=True)
sns.boxplot(data=df, x="dept", y="salary")
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
sns.pairplot(df[["age","salary","score"]])
sns.countplot(data=df, x="dept")
sns.violinplot(data=df, x="dept", y="salary")
```

21 1 Quick Reference Summary Tables

21.1 1Google Sheets Function Summary

Function Quick Reference

Category	Key Functions	Example
Aggregate	SUM, AVG, COUNT, MIN, MAX	`=SUM(A:A)`
Conditional	COUNTIF, SUMIF, AVERAGEIF	`=SUMIF(B:B,"Sales",C:C)`
Lookup	VLOOKUP, XLOOKUP, INDEX-MATCH	`=XLOOKUP(A2,D:D,E:E)`
Logical	IF, AND, OR, IFS, IFERROR	`=IF(A1>100,"High","Low")`
Text	LEFT, RIGHT, MID, LEN, TRIM	`=TRIM(A2)`
Date	TODAY, DATEDIF, EOMONTH	`=DATEDIF(A1,B1,"D")`
Math	ROUND, ABS, MOD, SQRT	`=ROUND(A1,2)`

21.2 1NumPy vs Pandas Comparison

NumPy vs Pandas

Task	NumPy	Pandas
Data structure	ndarray	Series / DataFrame
Index	Integer position	Labelled (string/date)
Data types	Homogeneous	Mixed types
Missing data	np.nan	NaN / None with tools
Statistics	`np.mean(arr)`	`df["col"].mean()`
Filter	`arr[arr>5]`	`df[df["A"]>5]`
Shape	`arr.shape`	`df.shape`
Use case	Numerical computation	Data analysis / wrangling

Exam Tips

- **VLOOKUP** only searches rightward; use INDEX-MATCH or XLOOKUP for flexible lookup
- **Absolute references** ($A$1) are needed when copying formulas referencing fixed tables
- **SUMIFS/COUNTIFS** syntax: result range first, then criteria ranges and values alternately
- **DataFrame vs Series**: Single column selection returns Series; double brackets return DataFrame
- **axis=0** = rows (default), **axis=1** = columns in Pandas
- **inplace=True** modifies the DataFrame directly; otherwise returns a copy
- **Boolean masking** in NumPy & Pandas: use & (and), — (or), ˜ (not) — not Python keywords
- **Vectorised operations** in NumPy are much faster than Python loops
- **reset_index()** after groupby to convert result back to a regular DataFrame
- Always check **dtypes** before analysis — common issue is numbers stored as strings